

Problem 1: Input Validation (The Do-While Loop)

Context: You need to force a user to enter valid data before the program can continue. **Task:**

- Write a program that asks the user to enter a "Player Level" which must be between **1 and 10**.
- Use a **do-while loop** to request the input.
- If the user enters a number outside this range (e.g., 11 or -5), the program must immediately ask them to try again.
- The program cannot proceed until a valid number is entered.
- Once valid, print "Level Accepted".

Problem 2: Linear Search (The While Loop)

Context: Searching through data using manual iteration. **Task:**

- Initialize a static array with these exact values: {15, 22, 8, 19, 42}.
- Ask the user to input a **Target ID** to search for.
- Use a **while loop** (do not use a for loop) to iterate through the array.
- If the number is found, print its **index**.
- If the number is not found after checking all elements, print "-1".

Problem 3: Logical Size Management (Array Insertion)

Context: Managing a static array where you must prevent the program from crashing if it runs out of space. **Task:**

- Declare a static array with a **capacity** of 5 integers (`int arr[5]`). (Note: Reduced to 5 for this exercise so it is easier to test the "Full" condition).
- Initialize it with 3 values: {10, 20, 30}.
- Create a variable named `size` to track active elements (initialize to 3).
- Ask the user to input a **new number**.
- **Safety Check:** Before inserting, check if the `size` is less than the `capacity`.
 - **If there is space:** Insert the number at the correct index and increment `size`.
 - **If the array is full:** Print "Error: Array is full" and **do not** insert the number.
- Print the final `size` to confirm.